

From source code to a supported developer workflow

A Kubernetes platform design and local MVP
for the Testkube sample application.

01 / DEVELOP	02 / DELIVER	03 / OPERATE
Edit source. Get fast feedback.	Build, deploy and test through one workflow.	Diagnose failures. Recover deliberately.

Design stance

Start with a reliable path for one team. Make production responsibilities explicit. Add platform components when a demonstrated need justifies their operational cost.

Includes requirements analysis, architecture, ownership, delivery controls, tradeoffs and an incremental roadmap. The repository contains the executable MVP, developer guide and separate validation evidence.

1. Recommendation

Give the application team a supported path from source code to a tested Kubernetes deployment: a few familiar commands, application-oriented feedback, and documented recovery steps. Start with versioned containers, declarative manifests, and CI acceptance tests. Add a shared delivery control plane when a shared environment exists.

The platform is a product for developers. Its success is whether a new developer can deploy, understand a failure, and recover without a platform engineer translating Kubernetes objects for them. Kubernetes adoption itself is not the outcome.

This repository implements a local platform using **kind + Helm + Make**, with a React frontend, Express API, and PostgreSQL. The accompanying CI workflow recreates the environment and runs acceptance checks. The proposed production architecture uses a managed Kubernetes service and managed PostgreSQL, with Git-based promotion and a reconciler such as Argo CD. Those production components are a design, not an implemented claim.

2. Brief, assumptions, and questions

The case asks for deployment tooling, a strong developer experience, and a presentation or document with critical analysis. An MVP is encouraged. The proposal distinguishes case requirements, explicit assumptions, implemented capabilities, and production follow-up.

The following are explicit assumptions, not facts established by the case:

Assumption	Why it matters	What would change the design
One application team and one small service boundary	A repository workflow is a sufficient first interface	Many teams justify service templates and a catalog
GitHub is the collaboration and CI system	Reuses the repository's existing home	Use equivalent controls in the organization's CI
A local, single-user demonstration is acceptable	kind provides an inexpensive acceptance environment	Shared preview environments need identity, quotas and expiry
Initial traffic is modest; no latency or uptime commitment exists	Resource values are starting estimates	Load tests and business SLOs determine capacity
Production data could matter even though this sample only reads a constant	Database recovery is a launch concern	Data classification determines encryption, retention and access
A managed cloud service is available for production	Avoids operating the control plane and database ourselves	Regulatory or hosting constraints may require operators and more staffing

Before production, confirm: who uses the app, who is on call, budget and hosting constraints, data classification, expected traffic, uptime expectations, and acceptable data loss and restoration time. Resolve these with product and engineering owners rather than inventing requirements to justify tools.

3. What the repository tells us

The application is a small three-tier system, but its original deployment contract was incomplete:

- The browser called `http://localhost:8080`; outside the developer laptop this points at the user's machine.
- The web Dockerfile used Vite's preview server. The platform preserves that developer-owned packaging.
- The API embedded database credentials in a connection URL and opened a client for each query. Failed database requests still returned HTTP 200.
- The API started listening when imported by tests and attached graceful shutdown handling to the Express app instead of the HTTP server.
- Existing Kubernetes examples deployed only the API. The ingress referenced a different Service name, and the image came from a temporary registry.
- The original API test checked only an HTTP success response; the browser test checked only a counter. Neither demonstrated that all three tiers worked together.

The MVP preserves all files under `apps/`, the root npm manifests and Docker Compose exactly as on `main`. Deployment tooling adapts to the existing contract: two local tunnels, fixed development database credentials, existing-route probes, and writable runtime volumes. Acceptance tests live outside the application tree. Proposed application improvements require a separately agreed developer-owned change; infrastructure must not silently redefine application behavior.

4. Architecture and developer journey

Local MVP

```
Developer -- make up --> dedicated kind cluster
      |               |
      +-- Docker builds --> local images loaded into kind

Browser: localhost:4173 --> web tunnel --> Vite preview + React
Browser: localhost:8080 --> API tunnel --> Express --> PostgreSQL
                                   |
                                   1 GiB PVC
```

Both tunnels bind only to loopback. The original frontend hardcodes `localhost:8080` and the API enables CORS. This supports a laptop demonstration; it does not provide an environment-independent production frontend. Helm supplies an init container to copy the web project to a writable volume for Vite's generated configuration, while preserving the original Dockerfiles and startup commands.

A new developer installs the documented prerequisites, runs `make up`, then `make open`. The application appears at one URL. `make deploy` rebuilds and deploys edits; `make check` verifies the complete request path; `make logs` and `make diagnose` explain operational failures. For rapid source edits, Vite and nodemon provide hot reload against a local development database.

The shared application chart in the tagged `platform-tools` package contains one Deployment and Service. It is installed independently as releases `api` and `web`, using per-service values in `deploy/`. PostgreSQL is release `db`, provisioned by a separately version- and checksum-pinned upstream chart; it is not an application chart dependency. Schema-validated application values configure image, replicas, port, probes, resources and runtime storage. Helm manages upgrades and revision history; the wrapper builds and loads local images and selects their references without modifying tracked files. Every Kubernetes command uses a dedicated kubeconfig and explicit context. The script does not depend on whichever cluster the developer happened to select earlier.

Production direction

```
Pull request -> unit + build + kind integration + browser checks
              -> review -> merge -> build once -> registry image digests
              -> staging environment PR -> Git reconciler -> staging
              -> smoke checks + approval -> production environment PR
              -> Git reconciler -> managed Kubernetes
```

```
Users -> TLS gateway -> web -> API -> managed PostgreSQL
                        |           |
                        logs/metrics backups + tested restore
```

An environment repository records reviewed image digests and configuration. CI publishes artifacts but does not hold a long-lived production kubeconfig. A reconciler applies the reviewed desired state. Production rollback reverts the environment commit, followed by verification; emergency changes must be reconciled back into Git. Database migration compatibility remains a separate responsibility.

5. Critical requirements and tradeoffs

Developer experience and ownership

Priority: essential. The team is unfamiliar with Kubernetes, so exposing a large chart configuration or asking developers to learn cluster administration would transfer platform work to them. Commands should describe tasks, fail clearly, and link to a runbook. The underlying YAML remains readable as an escape hatch.

Make is broadly available and sufficient for this service. Its limitations are shell portability, installation friction, and less structured validation. If repeated onboarding failures emerge, graduate to a versioned CLI or dev container. A portal comes after a useful workflow; otherwise it merely wraps unreliable steps in a UI.

The application team owns application behavior, dependencies, schema changes, and service health. The platform team owns templates, cluster lifecycle, delivery controls, baseline security, and shared telemetry. Both own actionable alerts and incident practice. Platform changes need versioning and a migration path, not silent changes to every team's templates.

Versioned platform ownership

Reusable scripts, the shared Helm chart, locked chart-test dependencies and the reusable CI workflow are owned in `taskovskig/platform-tools`. This repository keeps a thin Makefile/bootstrap, declarative service configuration, per-service values and application-specific acceptance tests. Both application Dockerfiles remain developer-owned.

`platform.lock.json` selects a semantic-version Git tag and manifest checksum. The bootstrap verifies the manifest and every packaged file before execution, caches by checksum, and fails on missing or changed tag content. CI references the same tag explicitly. Platform upgrades therefore become application review decisions rather than silent changes. Published tags must remain immutable. Local checkout overrides support pre-release testing and are prohibited in CI.

Separating repositories introduces a release-order dependency: publish the platform tag before the consumer PR can pass hosted CI. It also adds bootstrap ownership and compatibility work. A shared chart is bundled with its runtime release to avoid independently selecting incompatible versions; environment values and application behavior remain in the application repository.

Correct deployment and feedback

Priority: essential. A green pod is insufficient evidence that the application works. The MVP verifies HTML delivery, API output, a real SQL query, and browser interactions. The same deployment script runs locally and in CI, reducing a class of environment differences.

Helm provides a versioned application package, a constrained values interface and standard release history and rollback. The chart keeps health checks and security defaults in templates rather than exposing every Kubernetes field. Template maintenance and one additional client tool are costs we accept for consistent packaging. `helm template` preserves inspectability, and schema plus rendered-resource tests catch configuration mistakes. The upstream repository's separate chart packages Testkube workflows; it does not supply this application deployment. `kind` is a convenient disposable integration environment, but does not reproduce a cloud load balancer, multi-zone storage, IAM, or production failure modes.

Application images get a unique local release tag. Dependencies use the committed npm lockfile; the `kind` node uses a digest. The developer Dockerfiles use the floating `node:lts` base and retain development dependencies; PostgreSQL uses a version-line tag, so builds are **not fully immutable**. Production requires digest pinning, automated update PRs, retained registry artifacts, image scanning and provenance. The repository's old dependencies also need a deliberate upgrade and vulnerability triage effort; packaging them in a container does not resolve that risk.

Availability and safe releases

Priority: essential, with production targets to agree. API startup, readiness and liveness all use the existing `/hello` route. They prove process responsiveness only. The original API stays Ready during a database outage and returns HTTP 200 with a failure message. Smoke tests therefore assert the exact SQL response, and the outage drill records this limitation instead of claiming database-aware readiness.

Stateless rollouts request zero unavailable replicas with one surge pod. This requires spare capacity and does not promise zero downtime. The original shutdown handler is attached to the Express app rather than its HTTP server; reliable connection draining is not established. Production requires developer-owned fixes for health, status codes and shutdown, plus traffic-under-rollout testing.

The local rollback command selects `SERVICE=api` or `SERVICE=web` and restores that release's prior Helm revision, including its manifests, values and image reference. The other application and database releases are untouched. It needs retained images. Deploying both applications is sequential, not atomic. Database upgrades are an explicit separate operation. PostgreSQL data, schema, external Secrets and volume resizing are not rolled back. Production needs backward-compatible API changes and expand/contract migrations, plus an explicit owner for deciding between rollback and a forward fix.

Data durability and recovery

Priority: essential before real data. A local PostgreSQL StatefulSet and PVC demonstrate persistence across pod replacement. They do not provide high availability, off-machine backups, or recovery after cluster deletion. The sample's SQL query reads a constant; a dedicated persistence drill is necessary to prove storage behavior.

For production, choose managed PostgreSQL with automated backups, point-in-time recovery, encryption, monitoring, and a tested restoration procedure. Kubernetes can run databases, but the operational burden needs a reason. Separate the migration role from the application's least-privileged database role; the MVP's initialization user is intentionally a local-only simplification.

No recovery target is implied by the brief. A proposed discussion starting point is RPO of 15 minutes and RTO of one hour. Product and operations must accept or replace those targets, select a service tier, and prove them through restoration drills before promising them.

Security and environment boundaries

Priority: essential for shared environments. The MVP runs containers as non-root, drops Linux capabilities, uses read-only root filesystems with specific writable volumes, disables service-account token mounts, and enforces the restricted pod-security profile. Both local tunnels bind to loopback. The API hardcodes public development credentials, so PostgreSQL must use api-user/api-password/api-db. A Secret configures PostgreSQL but does not make those known credentials confidential. Local platform state is excluded from the allowlisted Docker build context.

These controls do not establish tenant isolation. The default kind network is not presented as a policy enforcement solution. A shared platform needs a CNI that enforces NetworkPolicy, default-deny rules with tested DNS/web/API/database allowances, scoped RBAC, quotas, workload identity, and external secret management. Use TLS and an agreed authentication model for public access. Production and development should have separate access and failure boundaries; namespaces alone are insufficient for hostile tenants.

Secrets in Kubernetes are not protected merely because they are encoded. Restrict reads, configure encryption at rest, audit access, and rotate credentials with the database. Updating a Secret without changing the actual database password can break the application. Retained PVCs must never be silently paired with a regenerated password.

Operability and cost

Priority: basic visibility now, service-level monitoring before production. The MVP provides stdout logs, deployment events, existing-route probes, rollout timeouts and a troubleshooting guide. It does not claim centralized logs, metrics dashboards, tracing, or alerting.

In a shared environment, measure request rate, error rate and duration; database connection pressure; pod restarts; and saturation. Associate release identity with telemetry. Agree an SLO and alert on sustained user impact and error-budget burn, with links to a runbook. Add tracing when a multi-service request path makes it useful, not because Kubernetes requires it.

Resource requests and limits are estimates, not capacity measurements. Measure under representative load, then tune. Proposed production web/API replicas should span failure domains; disruption budgets and autoscaling must follow actual availability and traffic requirements. Database capacity and connection limits can be the scaling constraint. Local kind keeps the demo inexpensive; cloud spend should be estimated from the agreed region, node footprint, database tier, backups and telemetry retention.

6. Delivery, acceptance, and scope

Capability	MVP status	Production follow-up
Independent application releases	Tagged platform chart; separate api and web values	Managed cluster and environment overlays
Developer commands and runbook	Implemented	Onboarding measurement and template versioning
Reproducible acceptance environment	kind, locked npm dependencies, versioned images	Immutable base digests and artifact provenance
CI quality gate	Workflow committed	Hosted execution and required branch checks
Health and deployment feedback	Probes, rollout waits, smoke and browser tests	SLOs, telemetry and alerting
Database persistence	Separate db release and local PVC	Managed database, backup/restore evidence
Credential handling	Fixed development credentials; production blocker	External store, rotation and scoped database role
Release recovery	Helm release revisions	Git-based promotion, retention, migration policy
Security baseline	Non-root restricted workloads	RBAC, enforced network policy, TLS, identity

The acceptance workflow has read-only repository permissions and creates a disposable cluster. It runs unit tests, a production frontend build, container builds, Kubernetes deployment, HTTP smoke tests, and browser tests, with diagnostic output on failure and cluster cleanup. No deployment to an external account is performed. Hosted CI execution must be verified separately; a committed workflow is not evidence that a GitHub run passed.

See `VALIDATION.md` for observed results and remaining limitations. Evidence should always distinguish a manifest that renders, a container that builds, a pod that is ready, and a browser journey that works.

7. Incremental platform roadmap

- 1. Prove the local path.** Complete onboarding with a developer unfamiliar with Kubernetes. Observe failures and improve instructions. Target an initial deployment within 15 minutes after prerequisites and cached downloads, as a hypothesis to measure rather than a claimed result.
- 2. Agree the application contract.** Request configurable credentials and database name, relative or configurable API URLs, production web serving, correct failure status, database-aware readiness and graceful shutdown. These are separate application changes, not included here.
- 3. Establish a shared staging environment.** Provision cloud infrastructure as code, managed PostgreSQL, a registry, TLS entry point, identity and Git reconciliation. Pin and scan images. Require CI checks and test migrations and restoration.
- 4. Meet production readiness criteria.** Agree SLO/RPO/RTO and ownership; test rollback, backup recovery, access restrictions, capacity and meaningful failure scenarios. Measure the actual recovery times.

5. **Scale the product based on demand.** Add preview environments with quotas and expiry, reusable service templates, and a service catalog when more teams need them. Consider Testkube if centrally scheduled or cross-environment test orchestration becomes a real need; the repository name alone does not justify installing it.

Track onboarding time, deployment lead time, failed-deployment recovery time, change failure rate, and developer support requests. Compare outcomes before and after adoption. Keep a short feedback loop with developers so the platform removes recurring work instead of accumulating features nobody needs.

8. Interview discussion guide

Present the problem and assumptions first, then show the running app and one deployment. Explain why application ownership, honest readiness signals and data recovery matter more than adding a catalog. Walk through a database outage and recovery, and identify which production responsibilities the demo intentionally leaves open.

Likely questions: Why Kubernetes for a small app? It is a case constraint; outside the case, compare a managed application runtime. Why Helm? It packages the application and provides standard release history while Make keeps the developer commands simple. Why not reuse the upstream chart? It contains Testkube workflow examples, not the application workloads. Why no service mesh? The demonstrated traffic and identity requirements do not require one. Why a database in kind but not the production cluster? Local repeatability and production recovery have different priorities. What would you build next? Shared staging with identity, safe artifact promotion and restoration evidence, based on the team's highest-risk unmet requirement.

References

- [kind quick start](#) and [v0.33.0 release](#): cluster setup, local image loading and the selected node image.
- [Kubernetes probe documentation](#): liveness, readiness and startup semantics.
- [Helm charts](#), [upgrade](#) and [rollback](#): packaging, values and release lifecycle.
- [Upstream workflow chart](#): Testkube examples, separate from the shared application chart implemented here.
- [PostgreSQL container documentation](#): initialization variables and persistent data directory behavior.

Database packaging: the [groundhog2k PostgreSQL chart](#) is pinned at 1.6.8 in the bootstrap. Its upgrade lifecycle is independent of both application releases. Production would replace this local release with separately provisioned managed PostgreSQL.